

智能法律助手 - 项目安装部署文档

📄 文档说明

本文档为智能法律助手项目的完整安装部署指南，适用于团队新成员或首次部署该项目的开发者。文档包含详细的软件安装步骤、数据库配置、环境变量设置等内容。

文档版本: v1.0
最后更新: 2026-01-22

📖 目录

- 1. [系统要求](#)
- 2. [必备工具软件安装](#)
- 3. [Docker安装](#)
- 4. [PostgreSQL数据库安装](#)
- 5. [Milvus向量数据库安装](#)
- 6. [MinIO对象存储安装](#)
- 7. [项目部署](#)
- 8. [验证部署](#)
- 9. [常见问题排查](#)

1. 系统要求

1.1 硬件要求

配置项	最低配置	推荐配置
CPU	2核	4核及以上
内存	8GB	16GB及以上
硬盘	40GB可用空间	100GB SSD
网络	宽带连接	宽带连接 (需要访问API服务)

1.2 操作系统支持

- **Windows 10/11** (推荐使用WSL2)
- **macOS 10.15+**
- **Linux Ubuntu 20.04+** (推荐)
- **Linux CentOS 7+**

1.3 软件依赖版本

软件	最低版本	推荐版本
----	------	------

软件	最低版本	推荐版本
Git	2.0+	2.30+
Python	3.9+	3.11
Node.js	16+	18 LTS
Docker	20.10+	24.0+
Docker Compose	2.0+	2.20+

2. 必备工具软件安装

2.1 Git 安装

Windows系统

1. 下载Git安装包

- 访问 [Git官网](#)
- 下载 Windows 版本安装包

2. 安装Git

```
# 运行下载的安装程序
# 使用默认配置一路点击 "Next" 完成安装
```

3. 验证安装

```
git --version
# 应显示: git version 2.x.x
```

4. 配置Git (首次使用时)

```
git config --global user.name "你的姓名"
git config --global user.email "你的邮箱"
```

macOS系统

```
# 使用Homebrew安装
brew install git

# 或使用Xcode命令行工具
```

```
xcode-select --install

# 验证安装
git --version
```

Linux系统 (Ubuntu/Debian)

```
# 更新软件包列表
sudo apt update

# 安装Git
sudo apt install git -y

# 验证安装
git --version

# 配置Git ( 首次使用 )
git config --global user.name "你的姓名"
git config --global user.email "你的邮箱"
```

Linux系统 (CentOS/RHEL)

```
# 安装Git
sudo yum install git -y
# 或使用dnf (CentOS 8+)
sudo dnf install git -y

# 验证安装
git --version
```

2.2 Python 安装

Windows系统

1. 下载Python安装包

- 访问 [Python官网](#)
- 下载 Python 3.11.x Windows installer

2. 安装Python

- 运行安装程序
- **重要:** 勾选 "Add Python to PATH" 选项
- 点击 "Install Now"

3. 验证安装

```
# 打开命令提示符或PowerShell
python --version
# 或
python3 --version

# 检查pip
pip --version
```

macOS系统

```
# 使用Homebrew安装 ( 推荐 )
brew install python@3.11

# 验证安装
python3 --version
pip3 --version

# 创建别名 ( 可选 )
echo "alias python='python3'" >> ~/.zshrc
echo "alias pip='pip3'" >> ~/.zshrc
source ~/.zshrc
```

Linux系统 (Ubuntu/Debian)

```
# 更新软件包列表
sudo apt update

# 安装Python 3.11
sudo apt install software-properties-common -y
sudo add-apt-repository ppa:deadsnakes/ppa -y
sudo apt update
sudo apt install python3.11 python3.11-venv python3.11-dev -y

# 安装pip
sudo apt install python3-pip -y

# 验证安装
python3.11 --version
pip3 --version

# 设置Python3.11为默认版本 ( 可选 )
sudo update-alternatives --install /usr/bin/python3 python3 /usr/bin/python3.11
1
```

Linux系统 (CentOS/RHEL)

```
# CentOS 7/8安装Python 3.11
sudo yum install gcc openssl-devel bzip2-devel libffi-devel zlib-devel -y
cd /usr/src
wget https://www.python.org/ftp/python/3.11.0/Python-3.11.0.tgz
tar xzf Python-3.11.0.tgz
cd Python-3.11.0
./configure --enable-optimizations
make altinstall

# 验证安装
python3.11 --version
```

2.3 Node.js 安装

Windows系统

1. 下载Node.js安装包

- 访问 [Node.js官网](https://nodejs.org/)
- 下载 18 LTS 版本安装包

2. 安装Node.js

- 运行安装程序
- 使用默认配置一路点击 "Next"

3. 验证安装

```
# 打开命令提示符或PowerShell
node --version
# 应显示: v18.x.x

npm --version
# 应显示: 9.x.x 或 10.x.x

# 配置npm国内镜像源 ( 可选 · 加速下载 )
npm config set registry https://registry.npmmirror.com
```

macOS系统

```
# 使用Homebrew安装
brew install node@18
```

```
# 验证安装
node --version
npm --version

# 配置npm镜像源 (可选)
npm config set registry https://registry.npmirror.com
```

Linux系统 (Ubuntu/Debian)

```
# 使用NodeSource仓库安装 (推荐)
curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install nodejs -y

# 验证安装
node --version
npm --version

# 配置npm镜像源 (可选)
npm config set registry https://registry.npmirror.com
```

Linux系统 (CentOS/RHEL)

```
# 使用NodeSource仓库安装
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
sudo yum install nodejs -y

# 验证安装
node --version
npm --version
```

3. Docker安装

3.1 Windows系统

注意: Windows上建议使用WSL2后端运行Docker

1. 安装WSL2

```
# 以管理员身份打开PowerShell
wsl --install

# 重启电脑后完成安装
```

2. 下载Docker Desktop

- 访问 [Docker官网](#)
- 下载 Windows 版本安装包

3. 安装Docker Desktop

- 运行安装程序
- 确保勾选 "Use WSL 2 based engine"
- 安装完成后重启电脑

4. 验证安装

```
# 打开PowerShell或命令提示符
docker --version
# 应显示: Docker version 24.x.x

docker-compose --version
# 应显示: Docker Compose version v2.x.x

# 测试Docker运行
docker run hello-world
```

3.2 macOS系统

1. 下载Docker Desktop for Mac

- 访问 [Docker官网](#)
- 下载 Mac 版本安装包

2. 安装Docker Desktop

```
# 打开下载的DMG文件
# 将Docker拖拽到Applications文件夹
# 启动Docker Desktop
```

3. 验证安装

```
docker --version
docker-compose --version

# 测试Docker运行
docker run hello-world
```

3.3 Linux系统 (Ubuntu/Debian)

```
# 1. 更新软件包索引
sudo apt update

# 2. 安装必要的依赖
sudo apt install apt-transport-https ca-certificates curl gnupg lsb-release -y

# 3. 添加Docker官方GPG密钥
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/docker-archive-keyring.gpg

# 4. 添加Docker仓库
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null

# 5. 更新软件包索引
sudo apt update

# 6. 安装Docker Engine
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin -y

# 7. 启动Docker服务
sudo systemctl start docker
sudo systemctl enable docker

# 8. 将当前用户添加到docker组 (避免每次使用sudo)
sudo usermod -aG docker $USER

# 9. 重新登录或执行以下命令使更改生效
newgrp docker

# 10. 验证安装
docker --version
docker compose version

# 测试Docker运行
docker run hello-world
```

3.4 Linux系统 (CentOS/RHEL)

```
# 1. 安装必要的依赖
sudo yum install -y yum-utils

# 2. 添加Docker仓库
sudo yum-config-manager --add-repo
https://download.docker.com/linux/centos/docker-ce.repo

# 3. 安装Docker Engine
```



```
sudo yum install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin -y

# 4. 启动Docker服务
sudo systemctl start docker
sudo systemctl enable docker

# 5. 将当前用户添加到docker组
sudo usermod -aG docker $USER

# 6. 重新登录
newgrp docker

# 7. 验证安装
docker --version
docker compose version

# 测试Docker运行
docker run hello-world
```

4. PostgreSQL数据库安装

本项目提供独立的PostgreSQL安装配置，包含pgvector扩展支持。

4.1 使用独立Docker Compose安装

项目已包含独立的PostgreSQL配置文件 `installation/pgvector/docker-compose.yml`。

配置说明：

- 镜像：`pgvector/pgvector:pg16` (PostgreSQL 16 + pgvector扩展)
- 数据库名：`legal_assistant`
- 用户名：`legal_assistant`
- 密码：`legal_assistant_123456`
- 端口：`5432`

4.2 启动PostgreSQL

使用独立的**docker-compose**文件启动

```
# 进入PostgreSQL安装目录
cd installation/pgvector

# 启动PostgreSQL服务
docker-compose up -d

# 查看容器状态
docker-compose ps
```

```
# 查看日志
docker-compose logs -f
```

启动成功输出示例

```
Creating network "pgvector_legal_network" with the default driver
Creating legal_assistant_db ... done
```

4.3 初始化数据库

重要：PostgreSQL容器启动后，需要手动执行数据库初始化脚本。

```
# 确保已在项目根目录 ( 如果不是，请先返回根目录 )
cd ../../ # 从 installation/pgvector 返回项目根目录

# 执行数据库schema初始化脚本
docker exec -i legal_assistant_db psql -U legal_assistant -d legal_assistant <
database/schema.sql

# 验证表已创建
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c
"\dt"

# 应看到以下表：
# users, conversations, messages, documents, document_embeddings
```

初始化成功输出示例：

```
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
```

4.4 验证PostgreSQL安装

```
# 检查容器状态
docker ps | grep legal_assistant_db

# 检查数据库健康状态
docker exec legal_assistant_db pg_isready -U legal_assistant -d legal_assistant
```

```

# 连接到数据库
docker exec -it legal_assistant_db psql -U legal_assistant -d legal_assistant

# 在psql中执行以下命令验证
\conninfo          # 查看连接信息
\l                 # 列出所有数据库
\dt                # 列出所有表
\dx                # 查看已安装的扩展

# 应该看到pgvector扩展
#  extname | extversion
#  -----+-----
#  vector  | 0.x.x

# 退出psql
\q

```

4.5 PostgreSQL常用操作

```

# 备份数据库
docker exec legal_assistant_db pg_dump -U legal_assistant legal_assistant >
backup_$(date +%Y%m%d).sql

# 恢复数据库
cat backup.sql | docker exec -i legal_assistant_db psql -U legal_assistant
legal_assistant

# 查看数据库日志
docker logs legal_assistant_db -f

# 停止数据库
cd installation/pgvector
docker-compose stop

# 启动数据库
docker-compose start

# 重启数据库
docker-compose restart

# 删除数据库容器和数据（谨慎使用！）
docker-compose down -v

# 重新创建数据库（会删除所有数据）
docker-compose down -v
docker-compose up -d

```

4.6 连接配置

在项目的 `.env` 文件中配置PostgreSQL连接：

```
# PostgreSQL数据库连接字符串
DATABASE_URL=postgresql+asyncpg://legal_assistant:legal_assistant_123456@localhost:5432/legal_assistant
```

4.7 使用图形化管理工具

可以使用以下工具连接数据库：

pgAdmin

- Host: `localhost`
- Port: `5432`
- Database: `legal_assistant`
- Username: `legal_assistant`
- Password: `legal_assistant_123456`

DBeaver

- Driver: PostgreSQL
- Host: `localhost`
- Port: `5432`
- Database: `legal_assistant`
- Username: `legal_assistant`
- Password: `legal_assistant_123456`

5. Milvus向量数据库安装

本项目提供独立的Milvus向量数据库安装配置。Milvus是高性能向量数据库，用于存储和检索文档的向量嵌入。

5.1 使用独立Docker Compose安装

项目已包含独立的Milvus配置文件 `installation/milvus/docker-compose.yml`。

配置说明：

- Milvus版本：`v2.3.3`
- 端口：`19530`（向量查询）、`9091`（监控API）
- 依赖服务：`etcd`、`MinIO`、`Pulsar`
- 数据存储：使用Docker volumes持久化

5.2 启动Milvus

```
# 进入Milvus安装目录
cd installation/milvus

# 启动Milvus服务
docker-compose up -d

# 查看容器状态
docker-compose ps

# 查看Milvus日志
docker-compose logs -f milvus

# 等待所有服务启动完成 ( 约30-60秒 )
```

启动成功输出示例

```
Creating network "milvus_milvus" with the default driver
Creating volume "milvus_milvus-etcd" with default driver
Creating volume "milvus_milvus-minio" with default driver
Creating volume "milvus_milvus-pulsar" with default driver
Creating volume "milvus_milvus-db" with default driver
Creating milvus_etcd    ... done
Creating milvus_minio   ... done
Creating milvus_pulsar  ... done
Creating milvus_standalone ... done
```

查看服务状态

所有服务启动成功后，应该看到以下容器运行：

```
docker ps
```

应该看到：

- **milvus_standalone** - Milvus主服务
- **milvus_etcd** - etcd配置存储
- **milvus_minio** - MinIO对象存储
- **milvus_pulsar** - Pulsar消息队列

5.3 验证Milvus安装

方法一：使用Python客户端验证

```

# 在Python虚拟环境中安装pymilvus
pip install pymilvus

# 创建验证脚本 test_milvus.py
cat > test_milvus.py << 'EOF'
from pymilvus import connections, utility

# 连接到Milvus
connections.connect(
    alias="default",
    host='localhost',
    port='19530'
)

# 检查连接状态
print("✅ 成功连接到Milvus")

# 列出所有集合
collections = utility.list_collections()
print(f"📋 当前集合列表: {collections}")

# 获取Milvus版本
from pymilvus import utility
try:
    print(f"🔗 Milvus版本信息获取成功")
except:
    print("⚠️ 无法获取版本信息")

# 关闭连接
connections.disconnect("default")
print("✅ 验证完成")
EOF

# 运行验证脚本
python test_milvus.py

# 删除临时脚本
rm test_milvus.py

```

方法二：使用curl验证API

```

# 检查Milvus健康状态
curl -X GET http://localhost:9091/healthz

# 应返回：OK

```

方法三：查看容器状态

```
# 查看所有Milvus相关容器
docker ps -a | grep milvus

# 查看Milvus容器日志
docker logs milvus_standalone

# 进入Milvus容器
docker exec -it milvus_standalone bash
```

5.4 Milvus常用操作

```
# 停止Milvus服务
cd installation/milvus
docker-compose stop

# 启动Milvus服务
docker-compose start

# 重启Milvus服务
docker-compose restart

# 查看服务日志
docker-compose logs -f

# 查看特定服务日志
docker-compose logs -f milvus
docker-compose logs -f etcd
docker-compose logs -f minio
docker-compose logs -f pulsar

# 停止并删除所有Milvus容器 ( 谨慎使用 ! )
docker-compose down

# 停止并删除所有Milvus容器和数据 ( 谨慎使用 ! 会删除所有向量数据 )
docker-compose down -v
```

5.5 Milvus数据备份与恢复

```
# 备份Milvus数据卷
docker run --rm \
  -v milvus_milvus-db:/data \
  -v $(pwd):/backup \
  ubuntu tar czf /backup/milvus_backup_$(date +%Y%m%d).tar.gz -C /data .

# 恢复Milvus数据卷
docker run --rm \
  -v milvus_milvus-db:/data \
```

```
-v $(pwd):/backup \
ubuntu tar xzf /backup/milvus_backup_YYYYMMDD.tar.gz -C /data
```

6. MinIO对象存储安装

MinIO用于存储文档文件。本项目在Milvus docker-compose中已包含MinIO服务，但也可以独立部署。

6.1 独立部署MinIO (可选)

```
# 创建MinIO数据目录
mkdir -p ~/minio/data

# 启动MinIO容器
docker run -d \
  --name legal_assistant_minio \
  -p 9000:9000 \
  -p 9001:9001 \
  -e "MINIO_ROOT_USER=uqDog1xApy0KOR0fVwx8" \
  -e "MINIO_ROOT_PASSWORD=xas1b6kc4Wz4G5vgUDKrp0lBsRaQ88MTzkl9EEa" \
  -v ~/minio/data:/data \
  minio/minio server /data --console-address ":9001"

# 验证MinIO
# 访问: http://localhost:9001
# 用户名: uqDog1xApy0KOR0fVwx8
# 密码: xas1b6kc4Wz4G5vgUDKrp0lBsRaQ88MTzkl9EEa
```

6.2 创建存储桶

```
# 使用mc命令行工具
docker run --rm --entrypoint /bin/sh minio/mc -c "
mc alias set myminio http://localhost:9000 uqDog1xApy0KOR0fVwx8
xas1b6kc4Wz4G5vgUDKrp0lBsRaQ88MTzkl9EEa
mc mb myminio/legal-documents
mc policy set download myminio/legal-documents
"
```

6.3 验证MinIO连接

```
# 测试MinIO API
curl http://localhost:9000/minio/health/live

# 应返回: OK
```



```
# 访问MinIO控制台
# 浏览器打开: http://localhost:9001
```

7. 项目部署

7.1 克隆项目代码

```
# 克隆项目仓库
git clone https://github.com/zhangkuntong/Intelligent_Legal_Assistant
cd Intelligent_Legal_Assistant

# 查看项目结构
ls -la
```

7.2 配置环境变量

```
# 复制环境变量模板
cp .env.example .env

# 编辑环境变量文件 (根据实际情况修改)
nano .env # 或使用 vim .env
```

关键配置项说明：

```
# ===== 应用基础配置 =====
ENVIRONMENT=development
HOST=0.0.0.0
PORT=8000

# ===== 数据库配置 =====
# PostgreSQL数据库连接字符串
DATABASE_URL=postgresql+asyncpg://legal_assistant:legal_assistant_123456@localhost:5432/legal_assistant

# ===== 安全配置 =====
# JWT密钥，生产环境必须修改为强密码
# 生成强密码命令: openssl rand -base64 32
SECRET_KEY=your-super-secret-key-change-in-production
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=10080

# ===== AI服务配置 =====
# 火山引擎API密钥 (必填)
LLM_API_KEY=d5ef8378-b9b6-4c76-98ee-c55ebda4954d
LLM_BASE_URL=https://ark.cn-beijing.volces.com/api/v3
```

```

LLM_MODEL=doubao-1-5-pro-32k-250115

# Embedding模型配置
EMBEDDING_MODEL_URL=https://ark.cn-beijing.volces.com/api/v3/embeddings/multimodal
EMBEDDING_MODEL=doubao-embedding-vision-250615

# ===== Milvus配置 =====
MILVUS_HOST=localhost
MILVUS_PORT=19530
MILVUS_COLLECTION_NAME=legal_documents
MILVUS_DIMENSION=2048

# ===== MinIO配置 =====
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=uqDog1xApy0KOR0fVwx8
MINIO_SECRET_KEY=xas1b6kc4Wz4G5vgUDKrp0lBsRaQ88MTzkgL9EEa
MINIO_SECURE=false
MINIO_BUCKET_NAME=legal-documents

# ===== 向量检索配置 =====
VECTOR_SEARCH_TOP_K=5
SIMILARITY_THRESHOLD=0.7

# ===== 文件上传配置 =====
UPLOAD_DIR=./uploads
MAX_FILE_SIZE=10485760
ALLOWED_FILE_TYPES=.pdf,.doc,.docx,.txt

# ===== Redis配置 (可选) =====
# REDIS_URL=redis://localhost:6379

# ===== 日志配置 =====
LOG_LEVEL=INFO
LOG_FILE=./logs/app.log

```

关键配置项说明：

```

# ===== 应用基础配置 =====
ENVIRONMENT=development
HOST=0.0.0.0
PORT=8000

# ===== 数据库配置 =====
# PostgreSQL数据库连接字符串
DATABASE_URL=postgresql+asyncpg://legal_assistant:legal_assistant_123456@localhost:5432/legal_assistant

# ===== 安全配置 =====
# JWT密钥，生产环境必须修改为强密码
# 生成强密码命令：openssl rand -base64 32

```

```

SECRET_KEY=your-super-secret-key-change-in-production
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=10080

# ===== AI服务配置 =====
# 火山引擎API密钥 (必填)
LLM_API_KEY=d5ef8378-b9b6-4c76-98ee-c55ebda4954d
LLM_BASE_URL=https://ark.cn-beijing.volces.com/api/v3
LLM_MODEL=doubao-1-5-pro-32k-250115

# Embedding模型配置
EMBEDDING_MODEL_URL=https://ark.cn-beijing.volces.com/api/v3/embeddings/multimodal
EMBEDDING_MODEL=doubao-embedding-vision-250615

# ===== Milvus配置 =====
MILVUS_HOST=localhost
MILVUS_PORT=19530
MILVUS_COLLECTION_NAME=legal_documents
MILVUS_DIMENSION=2048

# ===== MinIO配置 =====
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=uqDog1xApy0K0R0fVwx8
MINIO_SECRET_KEY=xas1b6kc4Wz4G5vgUDKrp0lBsRaQ88MTzKpL9EEa
MINIO_SECURE=false
MINIO_BUCKET_NAME=legal-documents

# ===== 向量检索配置 =====
VECTOR_SEARCH_TOP_K=5
SIMILARITY_THRESHOLD=0.7

# ===== 文件上传配置 =====
UPLOAD_DIR=./uploads
MAX_FILE_SIZE=10485760
ALLOWED_FILE_TYPES=.pdf,.doc,.docx,.txt

# ===== Redis配置 (可选) =====
# REDIS_URL=redis://localhost:6379

# ===== 日志配置 =====
LOG_LEVEL=INFO
LOG_FILE=./logs/app.log

```

7.3 启动所有服务

方式一：**Docker**容器部署（推荐用于快速启动和生产环境）

```

# 1. 启动PostgreSQL数据库（使用独立配置）
cd installation/pgvector

```

```

docker-compose up -d

# 2. 手动初始化数据库 ( 首次安装必须执行 )
cd ../..
docker exec -i legal_assistant_db psql -U legal_assistant -d legal_assistant <
database/schema.sql

# 3. 启动Milvus向量数据库 ( 使用独立配置 )
cd installation/milvus
docker-compose up -d

# 等待Milvus启动完成 ( 约30-60秒 )
docker-compose ps
cd ../..

# 4. 启动主服务 ( Redis、后端、前端 )
docker-compose up -d redis backend frontend

# 5. 查看所有服务状态
docker-compose ps

# 6. 查看服务日志
docker-compose logs -f

# 如果需要查看特定服务日志
docker-compose logs -f backend
docker-compose logs -f frontend
docker-compose logs -f redis

```

方式二：本地服务启动 (推荐用于开发和调试)

本地启动方式可以更方便地使用IDE的调试功能、查看实时日志、快速修改代码验证。

步骤1：启动基础服务 (使用Docker)

```

# 1. 启动PostgreSQL数据库 ( 使用独立配置 )
cd installation/pgvector
docker-compose up -d

# 2. 手动初始化数据库 ( 首次安装必须执行 )
cd ../..
docker exec -i legal_assistant_db psql -U legal_assistant -d legal_assistant <
database/schema.sql

# 3. 启动Milvus向量数据库 ( 使用独立配置 )
cd installation/milvus
docker-compose up -d
cd ../..

# 4. 启动Redis ( 使用Docker )

```

```
docker-compose up -d redis
```

5. 验证服务状态

```
docker ps
```

应该看到以下容器运行：

- legal_assistant_db (PostgreSQL)

- legal_assistant_redis (Redis)

- milvus_standalone (Milvus)

- milvus_etcd, milvus_minio, milvus_pulsar (Milvus依赖)

步骤2：启动后端服务（本地运行）

1. 进入后端目录

```
cd backend
```

2. 创建Python虚拟环境（首次运行）

Windows：

```
python -m venv venv
```

Linux/macOS：

```
python3 -m venv venv
```

3. 激活虚拟环境

Windows：

```
venv\Scripts\activate
```

Linux/macOS：

```
source venv/bin/activate
```

4. 升级pip

```
pip install --upgrade pip
```

5. 安装Python依赖

```
pip install -r requirements.txt
```

6. 创建必要的目录

```
mkdir -p uploads
```

```
mkdir -p logs
```

7. 启动后端服务

开发模式（支持热重载）

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

或使用Python直接运行

```
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

后端服务启动成功后，您会看到类似以下输出：

```
INFO:      Started server process [12345]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
🚀 智能法律助手后端服务启动完成
INFO:      Milvus服务初始化成功
INFO:      MinIO服务初始化成功
```

后端开发技巧：

- 访问 API 文档：<http://localhost:8000/docs>
- 访问健康检查：<http://localhost:8000/health>
- 使用 VS Code 调试：配置 `.vscode/launch.json`

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: FastAPI",
      "type": "python",
      "request": "launch",
      "module": "uvicorn",
      "args": [
        "app.main:app",
        "--host",
        "0.0.0.0",
        "--port",
        "8000",
        "--reload"
      ],
      "cwd": "${workspaceFolder}/backend",
      "envFile": "${workspaceFolder}/.env",
      "console": "integratedTerminal"
    }
  ]
}
```

步骤3：启动前端服务（本地运行）

```
# 1. 打开新的终端窗口（保持后端服务运行）

# 2. 进入前端目录
cd frontend

# 3. 安装Node.js依赖（首次运行）
npm install
```

```
# 4. 配置前端环境变量 ( 如需要 )
# 编辑 frontend/.env 文件
# 确保 VITE_API_BASE_URL=http://localhost:8000

# 5. 启动前端开发服务器
npm run dev

# 前端服务默认运行在 http://localhost:5173
# 如果需要修改端口，使用：
npm run dev -- --port 3000
```

前端服务启动成功后，您会看到类似以下输出：

```
VITE v4.x.x  ready in xxx ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

前端开发技巧：

- 在浏览器中访问：<http://localhost:5173>
- 使用 VS Code 配合 Vue DevTools 进行调试
- 代码修改后自动热重载，无需手动刷新

步骤4：验证本地服务

```
# 在新的终端窗口中执行验证命令

# 1. 验证后端服务
curl http://localhost:8000/health

# 应返回：
# {
#   "status": "healthy",
#   "timestamp": "2025-01-01T00:00:00Z"
# }

# 2. 验证API文档访问
# 浏览器打开：http://localhost:8000/docs

# 3. 验证前端服务
# 浏览器打开：http://localhost:5173

# 4. 验证数据库连接
docker exec legal_assistant_db pg_isready -U legal_assistant -d legal_assistant
```

```
# 5. 验证Milvus连接
docker exec milvus_standalone curl http://localhost:19530
```

步骤5：开发 workflow 建议

```
# 终端窗口1：运行后端
cd backend
venv\Scripts\activate # Windows
# source venv/bin/activate # Linux/macOS
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000

# 终端窗口2：运行前端
cd frontend
npm run dev

# 终端窗口3：监控日志 ( 可选 )
cd installation/milvus
docker-compose logs -f
cd ../..
docker-compose logs -f redis

# 终端窗口4：运行测试 ( 可选 )
cd backend
pytest
```

步骤6：停止本地服务

```
# 停止后端 ( 在后端终端窗口按 Ctrl+C )

# 停止前端 ( 在前端终端窗口按 Ctrl+C )

# 停止Docker服务
cd installation/pgvector
docker-compose stop
cd ../..
cd installation/milvus
docker-compose stop
cd ../..
docker-compose stop redis

# 如需完全停止并删除容器
cd installation/pgvector
docker-compose down
cd ../..
cd installation/milvus
docker-compose down
```



```
cd ../../  
docker-compose down redis
```

常见本地开发问题

问题1：Python依赖安装失败

```
# 清理缓存重试  
pip cache purge  
pip install -r requirements.txt --force-reinstall  
  
# 使用国内镜像源加速  
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

问题2：端口被占用

```
# Windows：查找占用端口的进程  
netstat -ano | findstr :8000  
# 结束进程：taskkill /PID <PID> /F  
  
# Linux/macOS：查找占用端口的进程  
lsof -i :8000  
# 结束进程：kill -9 <PID>
```

问题3：前端无法连接后端

- 检查 `frontend/.env` 中的 `VITE_API_BASE_URL` 是否正确
- 确保后端服务正在运行
- 检查浏览器控制台的网络请求错误
- 确认后端CORS配置允许前端源

问题4：数据库连接失败

```
# 检查PostgreSQL容器状态  
docker ps | grep legal_assistant_db  
  
# 测试数据库连接  
docker exec -it legal_assistant_db psql -U legal_assistant -d legal_assistant  
  
# 检查.env中的DATABASE_URL配置  
grep DATABASE_URL ../.env
```

7.4 等待服务启动

服务启动需要一些时间，请观察日志输出：

```
# 查看后端服务日志，等待看到 "Application startup complete"
docker-compose logs -f backend

# 查看前端服务日志
docker-compose logs -f frontend
```

正常启动后应该看到类似以下输出：

```
backend      | INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to
backend      | quit)
backend      | INFO:      Started server process [1]
backend      | 🚀 智能法律助手后端服务启动完成
backend      | INFO:      Milvus服务初始化成功
backend      | INFO:      MinIO服务初始化成功
```

7.5 首次启动的数据库初始化

重要提示：PostgreSQL使用独立配置安装后，必须手动执行数据库初始化脚本。

```
# 检查PostgreSQL容器是否正常运行
docker ps | grep legal_assistant_db

# 手动执行数据库初始化（首次安装必须执行）
docker exec -i legal_assistant_db psql -U legal_assistant -d legal_assistant <
database/schema.sql

# 验证表已创建
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c
"\dt"

# 应看到以下表：
# users, conversations, messages, documents, document_embeddings
```

8. 验证部署

8.1 检查所有服务状态

```
# 查看所有Docker容器
docker ps

# 应该看到以下容器在运行：
# - legal_assistant_db (PostgreSQL)
```

```
# - legal_assistant_redis (Redis)
# - legal_assistant_backend (后端服务)
# - legal_assistant_frontend (前端服务)
# - milvus_standalone (Milvus)
# - milvus_etcd (etcd)
# - milvus_minio (MinIO)
# - milvus_pulsar (Pulsar)
```

8.2 验证后端服务

```
# 健康检查
curl http://localhost:8000/health

# 应返回:
# {
#   "status": "healthy",
#   "timestamp": "2025-01-01T00:00:00Z"
# }

# 访问API文档
# 浏览器打开: http://localhost:8000/docs
```

8.3 验证前端服务

```
# 访问前端应用
# 浏览器打开: http://localhost:3000

# 应该看到智能法律助手登录页面
```

8.4 验证数据库连接

```
# 连接到PostgreSQL
docker exec -it legal_assistant_db psql -U legal_assistant -d legal_assistant

# 检查pgvector扩展
SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';

# 应返回:
#  extname | extversion
#  -----+-----
#  vector  | 0.x.x

# 检查向量表
\d document_embeddings
```

```
# 应看到包含VECTOR字段的表结构

# 退出
\q
```

8.5 验证Milvus连接

```
# 在Python环境中测试
python << 'EOF'
from pymilvus import connections, utility

# 连接到Milvus
connections.connect(alias="default", host='localhost', port='19530')

# 检查连接
print("☑ Milvus连接成功")

# 查看集合列表
collections = utility.list_collections()
print(f"📁 集合列表: {collections}")

connections.disconnect("default")
EOF
```

8.6 验证MinIO连接

```
# 测试MinIO API
curl http://localhost:9000/minio/health/live

# 应返回: OK

# 访问MinIO控制台
# 浏览器打开: http://localhost:9001
# 登录凭证在环境变量中配置
```

8.7 创建测试用户

```
# 使用API创建测试用户
curl -X POST http://localhost:8000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "testuser",
    "email": "test@example.com",
    "password": "Test123456",
    "full_name": "测试用户"
  }'
```

```
}'
```

```
# 应返回用户信息和token
```

9. 常见问题排查

9.1 Docker相关问题

问题1: Docker命令需要sudo

```
# 原因：当前用户不在docker组
# 解决：将用户添加到docker组
sudo usermod -aG docker $USER

# 重新登录或执行
newgrp docker
```

问题2: Docker容器无法启动

```
# 查看容器日志
docker logs <container_name>

# 例如：
docker logs legal_assistant_backend
docker logs milvus_standalone

# 检查端口占用
netstat -tlnp | grep <port>

# 检查Docker磁盘空间
docker system df

# 清理未使用的资源
docker system prune -a
```

问题3: docker-compose命令不存在

```
# 原因：docker-compose未安装或未在PATH中
# 检查版本
docker compose version # Docker Compose V2
docker-compose --version # Docker Compose V1

# 如果都不存在，安装Docker Compose
sudo curl -L
```

```
"https://github.com/docker/compose/releases/download/v2.20.0/docker-  
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose  
sudo chmod +x /usr/local/bin/docker-compose
```

9.2 PostgreSQL相关问题

问题1: 数据库连接失败

```
# 检查容器状态  
docker ps | grep legal_assistant_db  
  
# 检查数据库日志  
docker logs legal_assistant_db  
  
# 手动连接测试  
docker exec -it legal_assistant_db psql -U legal_assistant -d legal_assistant  
  
# 检查端口映射  
docker port legal_assistant_db  
  
# 检查网络连接  
telnet localhost 5432  
# 或  
nc -zv localhost 5432
```

问题2: pgvector扩展未安装

```
# 检查扩展  
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c  
"\dx"  
  
# 手动安装扩展  
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c  
"CREATE EXTENSION IF NOT EXISTS vector;"  
  
# 验证安装  
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c  
"SELECT extname, extversion FROM pg_extension WHERE extname = 'vector';"
```

问题3: 数据库表未创建

```
# 手动执行schema.sql  
docker exec -i legal_assistant_db psql -U legal_assistant -d legal_assistant <  
database/schema.sql
```

```
# 验证表创建
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c
"\dt"
```

9.3 Milvus相关问题

问题1: Milvus容器启动失败

```
# 检查依赖服务状态
cd installation/milvus
docker-compose ps

# 检查Milvus日志
docker logs milvus_standalone

# 检查etcd日志
docker logs milvus_etcd

# 检查minio日志
docker logs milvus_minio

# 检查pulsar日志
docker logs milvus_pulsar

# 查看所有服务日志
docker-compose logs -f
```

问题2: 无法连接到Milvus

```
# 检查端口是否开放
netstat -tlnp | grep 19530

# 检查Milvus健康状态
curl http://localhost:9091/healthz

# 检查容器网络
docker network ls
docker network inspect milvus_milvus

# 从容器内部测试连接
docker exec milvus_standalone curl http://localhost:19530
```

问题3: Milvus集合创建失败

```

# 检查Milvus日志
docker logs milvus_standalone | tail -100

# 使用Python测试连接
pip install pymilvus
python << 'EOF'
from pymilvus import connections

connections.connect(alias="default", host='localhost', port='19530')
print("连接成功")
EOF

# 如果连接成功，检查存储空间
docker exec milvus_standalone df -h

# 查看服务状态
cd installation/milvus
docker-compose ps

```

9.4 后端服务相关问题

问题1: 后端启动失败

```

# 查看后端日志
docker-compose logs backend

# 常见原因检查:

# 1. 数据库连接失败
# 检查DATABASE_URL配置
grep DATABASE_URL .env

# 2. Milvus连接失败
# 检查Milvus是否正常运行
docker ps | grep milvus

# 3. 环境变量缺失
# 检查.env文件
cat .env

# 4. 依赖安装问题
# 重新构建镜像
docker-compose build --no-cache backend
docker-compose up -d backend

```

问题2: API请求失败


```
# 检查后端健康状态
curl http://localhost:8000/health

# 检查后端日志
docker-compose logs -f backend

# 测试特定API端点
curl -X GET http://localhost:8000/api/users \
  -H "Authorization: Bearer YOUR_TOKEN"

# 检查CORS配置
# 在.env中检查CORS_ORIGINS
```

9.5 前端服务相关问题

问题1: 前端页面无法访问

```
# 检查容器状态
docker ps | grep frontend

# 检查前端日志
docker-compose logs frontend

# 检查端口
curl http://localhost:3000

# 重新构建前端
docker-compose build frontend
docker-compose up -d frontend
```

问题2: 前端无法调用API

```
# 检查API地址配置
# 前端配置中应设置: VITE_API_BASE_URL=http://localhost:8000

# 检查CORS配置
# 后端.env中应设置允许的源
cat .env | grep CORS_ORIGINS

# 检查浏览器控制台错误
# F12打开开发者工具查看Network和Console
```

9.6 性能和资源问题

问题1: 内存不足

```
# 查看容器资源使用
docker stats

# 限制容器资源使用
# 编辑docker-compose.yml添加:
services:
  milvus:
    mem_limit: 4g
    cpus: '2.0'

# 停止不必要的服务
docker-compose stop <service_name>
```

问题2: 磁盘空间不足

```
# 查看Docker磁盘使用
docker system df

# 清理未使用的镜像
docker image prune -a

# 清理未使用的容器
docker container prune

# 清理未使用的卷
docker volume prune

# 清理构建缓存
docker builder prune
```

9.7 日志调试

```
# 查看所有服务日志
docker-compose logs

# 实时查看日志
docker-compose logs -f

# 查看特定服务日志
docker-compose logs -f backend
docker-compose logs -f frontend
docker-compose logs -f postgres

# 查看最近100行日志
docker-compose logs --tail=100 backend
```

```
# 查看带时间戳的日志
docker-compose logs -t backend
```

10. 开发环境设置 (补充说明)

本章节是对第7.3节"方式二：本地服务启动"的补充说明，提供更多开发工具和技巧。

10.1 VS Code开发环境配置

推荐的**VS Code**扩展

```
{
  "recommendations": [
    "ms-python.python",
    "ms-python.pylance",
    "ms-python.vscode-pylance",
    "dbaeumer.vscode-eslint",
    "esbenp.prettier-vscode",
    "vue.volar",
    "vue.vscode-typescript-vue-plugin",
    "ms-azuretools.vscode-docker",
    "eamodio.gitlens",
    "formulahendry.auto-rename-tag",
    "christian-kohler.path-intellisense"
  ]
}
```

VS Code工作区配置 (**.vscode/settings.json**)

```
{
  "python.linting.enabled": true,
  "python.linting.pylintEnabled": false,
  "python.linting.flake8Enabled": true,
  "python.formatting.provider": "black",
  "python.testing.pytestEnabled": true,
  "python.testing.pytestArgs": [
    "tests"
  ],
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": true
  },
  "typescript.tsdk": "node_modules/typescript/lib",
  "vite.devServer.port": 5173
}
```

后端调试配置 (.vscode/launch.json)

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Python: FastAPI",
      "type": "python",
      "request": "launch",
      "module": "uvicorn",
      "args": [
        "app.main:app",
        "--host",
        "0.0.0.0",
        "--port",
        "8000",
        "--reload"
      ],
      "cwd": "${workspaceFolder}/backend",
      "envFile": "${workspaceFolder}/.env",
      "console": "integratedTerminal",
      "justMyCode": false
    },
    {
      "name": "Python: Current File",
      "type": "python",
      "request": "launch",
      "program": "${file}",
      "cwd": "${workspaceFolder}/backend",
      "envFile": "${workspaceFolder}/.env",
      "console": "integratedTerminal"
    },
    {
      "name": "Python: Pytest",
      "type": "python",
      "request": "launch",
      "module": "pytest",
      "cwd": "${workspaceFolder}/backend",
      "envFile": "${workspaceFolder}/.env",
      "console": "integratedTerminal"
    }
  ]
}
```

前端调试配置 (.vscode/launch.json 添加)

```
{
  "version": "0.2.0",
  "configurations": [
```

```
// ... Python配置 ...
{
  "name": "Vue.js: debug",
  "type": "chrome",
  "request": "launch",
  "url": "http://localhost:5173",
  "webRoot": "${workspaceFolder}/frontend/src",
  "breakOnLoad": true,
  "sourceMapPathOverrides": {
    "webpack://src/*": "${webRoot}/*"
  }
}
]
}
```

10.2 代码质量工具

后端代码格式化和检查

```
# 安装开发依赖
pip install black isort flake8 mypy

# 格式化代码
black app/ tests/
isort app/ tests/

# 代码检查
flake8 app/ tests/

# 类型检查
mypy app/

# 一次性运行所有检查
black app/ tests/ && isort app/ tests/ && flake8 app/ tests/
```

前端代码格式化和检查

```
# 安装ESLint和Prettier
npm install --save-dev eslint prettier eslint-config-prettier eslint-plugin-vue

# 运行ESLint
npm run lint

# 修复ESLint问题
npm run lint -- --fix
```

```
# 格式化代码
npx prettier --write "src/**/*.{vue,ts,js,css}"
```

10.3 测试工具

后端测试

```
# 安装测试依赖
pip install pytest pytest-asyncio pytest-cov httpx

# 运行所有测试
pytest

# 运行特定测试文件
pytest tests/test_api.py

# 运行特定测试函数
pytest tests/test_api.py::test_create_user

# 查看测试覆盖率
pytest --cov=app --cov-report=html

# 查看覆盖率报告
open htmlcov/index.html # macOS
start htmlcov/index.html # Windows
xdg-open htmlcov/index.html # Linux
```

前端测试 (如配置)

```
# 运行单元测试
npm run test:unit

# 运行端到端测试
npm run test:e2e

# 查看测试覆盖率
npm run test:coverage
```

10.4 数据库管理工具

推荐工具

1. pgAdmin - PostgreSQL图形化管理工具

- 下载 : <https://www.pgadmin.org/download/>
- 连接配置 :

- Host: localhost
- Port: 5432
- Database: legal_assistant
- Username: legal_assistant
- Password: legal_assistant_123456

2. DBeaver - 通用数据库管理工具

- 下载 : <https://dbeaver.io/download/>
- 支持PostgreSQL、Milvus等多种数据库

3. TablePlus - macOS轻量级数据库客户端

- 下载 : <https://tableplus.com/>

命令行数据库操作

```
# 连接到PostgreSQL
docker exec -it legal_assistant_db psql -U legal_assistant -d legal_assistant

# 常用psql命令
\l                # 列出所有数据库
\dt               # 列出所有表
\d <table_name>   # 查看表结构
\dv               # 列出所有视图
\du               # 列出所有用户
\conninfo         # 显示连接信息
\q               # 退出

# SQL查询示例
SELECT * FROM users LIMIT 10;
SELECT COUNT(*) FROM documents;
SELECT username, email FROM users WHERE is_active = true;

# 导出数据库
docker exec legal_assistant_db pg_dump -U legal_assistant legal_assistant >
backup.sql

# 导入数据库
docker exec -i legal_assistant_db psql -U legal_assistant legal_assistant <
backup.sql
```

10.5 API测试工具

使用Postman测试API

1. 安装Postman : <https://www.postman.com/downloads/>
2. 导入API定义 : 访问 <http://localhost:8000/openapi.json>
3. 配置环境变量 :

- `base_url`: `http://localhost:8000`
- `token`: 从登录接口获取

使用curl测试API

```
# 健康检查
curl http://localhost:8000/health

# 用户注册
curl -X POST http://localhost:8000/api/auth/register \
  -H "Content-Type: application/json" \
  -d '{
    "username": "testuser",
    "email": "test@example.com",
    "password": "Test123456",
    "full_name": "测试用户"
  }'

# 用户登录
curl -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "username": "testuser",
    "password": "Test123456"
  }'

# 获取用户列表 (需要token)
TOKEN="your_token_here"
curl http://localhost:8000/api/users \
  -H "Authorization: Bearer $TOKEN"
```

10.6 日志查看和分析

```
# 实时查看后端日志
tail -f backend/logs/app.log

# 查看错误日志
grep ERROR backend/logs/app.log

# 查看今天的日志
grep "$(date +%Y-%m-%d)" backend/logs/app.log

# 查看Docker容器日志
docker-compose logs -f backend
docker-compose logs --tail=100 backend

# 使用docker logs查看特定容器
docker logs legal_assistant_backend --tail 50 -f
```



```
# 导出日志
docker logs legal_assistant_backend > backend_logs.txt
```

10.7 性能分析

后端性能分析

```
# 安装性能分析工具
pip install py-spy

# 分析CPU使用情况
py-spy top --pid $(pgrep -f "uvicorn app.main:app")

# 生成性能分析报告
py-spy record -o profile.svg --pid $(pgrep -f "uvicorn app.main:app")

# 查看内存使用
docker stats legal_assistant_backend
```

数据库性能分析

```
# 查看数据库连接数
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c
"SELECT count(*) FROM pg_stat_activity;"

# 查看慢查询
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c
"SELECT query, mean_exec_time FROM pg_stat_statements ORDER BY mean_exec_time
DESC LIMIT 10;"

# 查看表大小
docker exec legal_assistant_db psql -U legal_assistant -d legal_assistant -c
"SELECT schemaname, tablename,
pg_size_pretty(pg_total_relation_size(schemaname||'.'||tablename)) FROM
pg_tables ORDER BY pg_total_relation_size(schemaname||'.'||tablename) DESC;"
```

10.8 开发工作流建议

典型的开发流程

```
# 1. 拉取最新代码
git pull origin main

# 2. 切换到新分支
git checkout -b feature/your-feature-name
```

```
# 3. 开发并提交代码
# ... 编写代码 ...
git add .
git commit -m "Add your feature description"

# 4. 运行测试
cd backend
pytest

# 5. 代码格式化
black app/ isort app/

# 6. 推送到远程
git push origin feature/your-feature-name

# 7. 创建Pull Request
# 在GitHub/GitLab上创建PR
```

Git提交规范

```
# feat: 新功能
git commit -m "feat: 添加用户头像上传功能"

# fix: 修复bug
git commit -m "fix: 修复文档上传时的内存泄漏问题"

# docs: 文档更新
git commit -m "docs: 更新API文档"

# style: 代码格式调整
git commit -m "style: 统一代码缩进格式"

# refactor: 重构
git commit -m "refactor: 重构向量检索服务"

# test: 测试相关
git commit -m "test: 添加用户认证单元测试"

# chore: 构建/工具链
git commit -m "chore: 更新Docker镜像版本"
```

10.2 VS Code开发环境

推荐的VS Code扩展：

- Python
- Pylance
- ESLint

- Vetur / Volar (Vue)
- Docker

11. 生产环境部署建议

11.1 安全配置

```
# 1. 修改所有默认密码
# 在.env中设置强密码

# 2. 修改SECRET_KEY
SECRET_KEY=$(openssl rand -base64 32)

# 3. 启用HTTPS
# 配置SSL证书

# 4. 限制端口访问
# 使用防火墙
sudo ufw enable
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw deny 8000/tcp # 限制后端直接访问
```

11.2 性能优化

```
# 1. 增加Milvus配置
# 调整索引参数

# 2. 数据库连接池优化
# 在backend配置中增加连接池大小

# 3. 启用Redis缓存
# 配置Redis连接
```

11.3 监控和日志

```
# 1. 配置日志轮转
# 2. 设置监控告警
# 3. 定期备份数据库
```

12. 卸载和清理

如果需要完全卸载项目：

```
# 1. 停止所有服务
cd installation/pgvector
docker-compose down
cd ../..
cd installation/milvus
docker-compose down
cd ../..
docker-compose down

# 2. 删除所有容器和数据 ( 谨慎使用 ! )
cd installation/pgvector
docker-compose down -v
cd ../..
cd installation/milvus
docker-compose down -v
cd ../..
docker-compose down -v

# 3. 删除Docker镜像
docker rmi $(docker images -q 'legal_assistant*')

# 4. 删除项目文件
cd ..
rm -rf Intelligent_Legal_Assistant
```

13. 技术支持

如有问题，请联系项目团队或查看以下资源：

- **API文档**: <http://localhost:8000/docs>
- **项目仓库**: [GitHub Repository URL]
- **Issue反馈**: [GitHub Issues URL]

14. 附录

14.1 默认端口列表

服务	端口	用途
前端	3000	Vue前端应用
后端API	8000	FastAPI后端服务
PostgreSQL	5432	PostgreSQL数据库
Redis	6379	Redis缓存
Milvus	19530	Milvus向量数据库

服务	端口	用途
Milvus Metrics	9091	Milvus监控API
MinIO API	9000	MinIO对象存储API
MinIO Console	9001	MinIO管理控制台
Nginx	80/443	Web服务器（生产环境）

14.2 默认凭据列表

服务	用户名	密码	用途
PostgreSQL	legal_assistant	legal_assistant_123456	数据库连接
MinIO	uqDog1xApy0KOR0fVwx8	xas1b6kc4Wz4G5vgUDKrpOIBsRaQ88MTzkpL9EEa	对象存储

⚠ **重要:** 生产环境请务必修改所有默认凭据！

14.3 目录结构说明

```

Intelligent_Legal_Assistant/
├── backend/                # 后端代码
│   ├── app/               # 应用代码
│   ├── requirements.txt    # Python依赖
│   └── Dockerfile          # 后端Docker镜像
├── frontend/              # 前端代码
│   ├── src/               # 源代码
│   ├── package.json        # Node.js依赖
│   └── Dockerfile          # 前端Docker镜像
├── database/              # 数据库脚本
│   ├── schema.sql          # 数据库结构
│   └── init.sql            # 初始化脚本
├── installation/          # 独立安装配置
│   ├── pgvector/          # PostgreSQL安装配置
│   │   └── docker-compose.yml # PostgreSQL独立部署
│   └── milvus/             # Milvus安装配置
│       └── docker-compose.yml # Milvus独立部署
├── docs/                  # 文档目录
├── docker-compose.yml      # 主服务编排
├── .env.example            # 环境变量模板
└── README.md              # 项目说明

```

文档结束

祝您部署顺利！如有任何问题，请参考本文档的"常见问题排查"章节。